"An Introduction to Prolog"
Part V

Viviana Mascardi
viviana.mascardi@unige.it

University of Genoa – DIBRIS

Dibris

Disclaimer

- This presentation may contain material protected by copyright laws.
- In particular, these lessons are based on the material that Peter Schachte made available on the Web years ago, as part of the Logic and Computation Course at the Computer Science Department of the University Melbourne
- We made this material available on the Web only to ensure timely dissemination among our students, and is meant only for students' personal use.
- Any use different from the students' personal use is prohibited.
- V. Mascardi thanks Professor Schachte for the courtesy of sharing his teaching material on Prolog.

Dibris

The univ predicate

Sometimes it is useful to be able to write predicates that will work on any kind of term

The =.. predicate, pronounced "univ," turns a term into a list

The first element of the list is the term's functor; following elements are the term's arguments, in order

```
?- p(1,q(3)) =.. X.
X = [p, 1, q(3)] ;
No

?- T =.. [p, 1, q(3)].
T = p(1, q(3)) ;
No
```

```
?- 1 =.. X.
X = [1] ;
No

?- [1,2,3] =.. X.
X = ['.', 1, [2, 3]] ;
No
```

The `functor` predicate

Because univ builds a list that is typically not needed, it is often better to use the separate builtin predicates `functor/3` and `arg/3`

`functor(Term,Name,Arity)` holds when the functor of `Term` is `Name` and its arity is `Arity`

For atomic terms, the whole term is considered to be the `Name`, and `Arity` is 0

Either the `Term` or both the `Name` and `Arity` must be bound, otherwise `functor` throws an exception

The functor predicate in action

```
?- functor(p(1,q(3)), F, A).
F = p
A = 2 ;
No

?- functor(T, p, 2).
T = p(_G280, _G281) ;
No

?- functor(1, F, A).
F = 1
A = 0 ;
No

?- functor(X, 42, 0).
X = 42 ;
No
```

The arg predicate

`arg/3` can be used to access the arguments of a term one at a time
`arg(N,Term,Arg)` holds if `Arg` is the Nth argument of `Term` (counting from `1`)
`Term` must be bound to a compound term
For most Prolog systems, `N` must be bound; SWI Prolog will backtrack over all arguments of term
`arg/3` is deterministic when first two arguments are bound

The arg predicate in action

```
?- arg(2, foo(a,b), Arg).
Arg = b ;

No
?- arg(N, foo(a,b), Arg).
N = 1
Arg = a ;

N = 2
Arg = b ;

No
?- arg(N, 3, 3).
ERROR: arg/3: Type error: ‘compound’ expected, found ‘3’
```

Using functor and arg

Collect a list of all the functors appearing anywhere in a ground term:

```
all_functors(Term, List) :- all_functors(Term, List, []).
all_functors(Term, List, List0) :-
        functor(Term, Name, Arity),
        (   Arity > 0 ->
            List = [Name|List1],
            all_functors1(Arity, Term, List1, List0)
        ;   List = List0
        ).
all_functors1(N, Term, List, List0) :-
        (   N =:= 0 ->
            List = List0
        ;   arg(N, Term, Arg),
            all_functors(Arg, List1, List0),
            N1 is N - 1,
            all_functors1(N1, Term, List, List1)
        ).
```

Using functor and arg (2)

```
?- all_functors(foo(bar([1,2],zip,baz(3)),bar(7)), L).
L = [foo, bar, '.', '.', baz, bar] ;
No

?- X=bar(42), all_functors(foo(X), L).
X = bar(42)
L = [foo, bar] ;
No

?- all_functors(foo(X), L), X=bar(42).
ERROR: Arguments are not sufficiently instantiated
   Exception: (11) functor(_G342, _G448, _G449) ? creep
```

Since `Name` and `Arity` are not bound when `functor/3` is called, that goal causes an error if `Term` is not bound.

Term Comparison

The sorting predicates work on lists of any kinds of terms
Built on general term comparison predicates
The predicates `@<`, `@=<`, `@>`, `@>=`, `==` `\==` work like `<`, `=<`, `>`, `>=`, `=:=` `=\=`, but they work on any terms
Atoms are compared lexicographically, numbers numerically
Compound terms are compared first by arity, then by functor, then by the arguments in order
Variables `@<` numbers `@<` atoms `@<` compound terms
Beware variables in terms being sorted!
`==` and `\==` check whether terms are identical — without binding any variables

Variable and Type Testing

Earlier we saw that the simple definition of `rev` used in the "backwards mode" goes into an infinite loop after finding the correct answer

```
rev([], []).
rev([A|BC], R) :-
        rev(BC, CB),
        append(CB, [A], R).
```

Recursive call to rev produces longer and longer reversed lists; once it has found the right length, no other length will be correct

Variable Testing Example

Can fix this by exchanging the calls in the clause body

```
rev([], []).
rev([A|BC], R) :-
        append(CB, [A], R),
        rev(BC, CB).
```

In this order, when `R` is bound, the call to `append/3` has only one solution, so it solves the problem
But now `rev/2` gets in trouble in the "forwards mode"
Once it finds the solution, asking for more solutions enters an infinite loop

Variable Testing

To implement a *reversible* version of `rev/2`, we need to decide which order to execute the goals based on which arguments are bound.
Prolog has a built in predicate `var/1` which succeeds if its argument is an unbound variable, and fails if not.

```
?- var(_).

Yes
?- var(a).

No
```

Other Non-logical Builtins

All of the following have the same problem of potentially making conjunction not commutative

`nonvar(X)` `X` is bound.

`ground(X)` `X` is bound, and every variable in the term it is bound to is also ground.

`atom(X)` `X` is bound to an atom.

`integer(X)` `X` is bound to an integer.

`float(X)` `X` is bound to a float.

`number(X)` `X` is bound to a number.

`atomic(X)` `atom(X) ; number(X).`

`compound(X)` `X` is bound to a compound term.

Defining `length/2`

We can define `length/2` to work when the list is supplied:

```
len1([], 0).
len1([_|L], N) :-
        len1(L, N1),
        N is N1 + 1.
```

and when the length is supplied:

```
len2([], 0).
len2([_|L], N) :-
        N > 0,
        N1 is N - 1,
        len2(L, N1).
```

Defining `length/2` (2)

A version that works in both modes:

```
len(L,N) :-
        (   var(N) ->
                len1(L,N)
        ;   len2(L,N)
        ).
```

NB: don't check if `L` is `var`: even if it's not, its tail may be!

Defining `length/2` (3)

```
?- len(L, 4).

L = [_G242, _G248, _G254, _G260] ;

No
?- len([a,b,c], N).

N = 3 ;

No
```

```
?- len(L, N).

L = []
N = 0 ;

L = [_G257]
N = 1 ;

L = [_G257, _G260]
N = 2

Yes
```

Introduction to program access and manipulation

- So far our programs have been assumed to be resident in the computer's memory, without discussion of how they are represented or how they got there.
- Many applications depend on accessing the clauses in the program.
- Furthermore, if programs are to be modified at runtime, there must be a way of adding (and deleting) clauses

Clause

- The system predicate for accessing the program is *clause(Head, Body)*.
- The goal *?- clause(Head, Body).* must be called with *Head* instantiated.
- The program is searched for the first clause whose head unifies with *Head*.
- The head and the body of this clause are then unified with Head and Body.
- On backtracking, the goal succeeds once for each unifiable clause in the predicate definition.
- Facts have the atom *true* as their body.
- **Clauses cannot be accessed via their body!**

Dibris

Assert

- The basic predicate for adding clauses to the program is *assert(Clause)* which adds *Clause* as the last clause of the corresponding predicate definition.
- There is a variant of *assert*, *asserta*, that adds the clause at the beginning of a predicate definition.
- If *Clause* is uninstantiated, an error condition occurs.

Retract

- The predicate *retract(Clause)* removes from the program the first clause in the program unifying with *Clause*.
- Note that to retract a clause such as `a :- b, c, d.` you need to specify `retract(a :- (b, c, d))` or `retract(a :- Body)`

Assert in action (1)

```
fib_print(0, 1).
fib_print(1, 1).
fib_print(N, F) :-
    write('Call of fib_print on '), write(N), nl,
    N > 1,
    N1 is N - 1,
    N2 is N - 2,
    fib_print(N1, F1),
    fib_print(N2, F2),
    F is F1 + F2.
```

Assert in action (2)

```
fib_stupid(0, 1).
fib_stupid(1, 1).
fib_stupid(N, F) :-
    write('Call of fib_stupid on '), write(N), nl,
    N > 1,
    N1 is N - 1,
    N2 is N - 2,
    fib_stupid(N1, F1),
    fib_stupid(N2, F2),
    F is F1 + F2,
    assert(fib_stupid(N, F)).
```

Assert in action (3)

```
fib_smart(0, 1).
fib_smart(1, 1).
fib_smart(N, F) :-
    write('Call of fib_smart on '), write(N), nl,
    N > 1,
    N1 is N - 1,
    N2 is N - 2,
    fib_smart(N1, F1),
    fib_smart(N2, F2),
    F is F1 + F2,
    asserta(fib_smart(N, F)).
```